

Redes neuronales: entrenamiento y diseño

Retropropagación, activaciones y arquitecturas

Curso de Inteligencia Artificial

Objetivos de la segunda sesión

- Derivar los gradientes de una red de dos capas.
- Actualizar todos los pesos y sesgos de un ejemplo.
- Distinguir pérdida, optimizador e hiperparámetros.
- Analizar activaciones, generalización y arquitecturas CNN, RNN y GAN.

Ruta de la segunda presentación

- Entrenamiento y función de pérdida.
- Gradientes y retropropagación paso a paso.
- Activaciones, inicialización y diseño experimental.
- Arquitecturas especializadas y ejercicios.

Ejemplo numérico: una red 2-2-1

Parámetros didácticos para calcular a mano:

$$x = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 1 & -1 \\ 0.5 & 0.5 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$W^{(2)} = [1 \quad -1], \quad b^{(2)} = 0$$

Usaremos sigmoide en ambas capas y una etiqueta conocida $y = 1$.

La arquitectura pequeña conserva el procedimiento de la red del capítulo.

Cálculo de la predicción

$$z^{(1)} = \begin{bmatrix} -0.5 \\ 0.75 \end{bmatrix}, \quad a^{(1)} = \begin{bmatrix} \sigma(-0.5) \\ \sigma(0.75) \end{bmatrix} \approx \begin{bmatrix} 0.3775 \\ 0.6792 \end{bmatrix}$$

$$z^{(2)} = 0.3775 - 0.6792 = -0.3017$$

$$\hat{y} = \sigma(-0.3017) \approx 0.4252$$

Con umbral 0.5, la red predice clase 0. La etiqueta $y = 1$ muestra que hay un error que el entrenamiento debe reducir.

Entrenamiento de la red

- Configurar arquitectura e inicializar parámetros.
- Calcular predicciones con propagación hacia adelante.
- Comparar predicciones con etiquetas mediante una pérdida.
- Obtener gradientes con retropropagación.
- Actualizar parámetros y repetir hasta el criterio de parada.

Fuente: capítulo 9, pp. 303–305.

Parámetros e hiperparámetros

	Parámetros	Hiperparámetros
Ejemplos	Pesos y sesgos	Capas, neuronas, tasa y épocas
Ajuste	Optimizador durante entrenamiento	Diseño y comparación con validación

Un entrenamiento ordinario mantiene fija la arquitectura mientras modifica sus parámetros.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Error de predicción y función de pérdida

El capítulo usa $y - \hat{y}$ como señal de error. Para explicar minimización distinguimos:

$$e = y - \hat{y} \quad (\text{error con signo})$$

$$L = \frac{1}{2}(\hat{y} - y)^2 \quad (\text{pérdida por ejemplo})$$

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)^2$$

La pérdida cuadrática evita que errores positivos y negativos se cancelen al agregarlos.

Fuente: capítulo 9, pp. 307 y 318. Distinción y notación didáctica.

Pérdida del ejemplo numérico

Para $y = 1$ y $\hat{y} \approx 0.4252$:

$$e = y - \hat{y} \approx 0.5748$$

$$L = \frac{1}{2}(\hat{y} - y)^2 \approx 0.1652$$

Si la predicción aumenta a 0.60:

$$L = \frac{1}{2}(0.60 - 1)^2 = 0.08$$

La pérdida disminuye porque la predicción se acerca a la etiqueta.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Descenso del gradiente

$$\theta_{\text{nuevo}} = \theta_{\text{actual}} - \eta \nabla_{\theta} L$$

- El gradiente indica cómo cambia la pérdida al modificar los parámetros.
- El signo negativo orienta el ajuste hacia una reducción local de la pérdida.
- $\eta > 0$ es la tasa de aprendizaje.
- Una superficie no convexa puede contener múltiples puntos estacionarios.

Fuente: capítulo 9, pp. 307–310.

Ejemplo de descenso en una dimensión

Sea $L(w) = (w - 2)^2$. Entonces $L'(w) = 2(w - 2)$.

Con $w = 0$ y $\eta = 0.1$:

$$w' = 0 - 0.1(-4) = 0.4$$

$$L(0) = 4, \quad L(0.4) = 2.56$$

Otra actualización produce $w'' = 0.72$.

La pendiente negativa indica que aumentar w reduce localmente la pérdida.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

La tasa de aprendizaje puede desestabilizar

Con la misma función y $\eta = 1.1$:

$$w = 0, \quad w' = 4.4, \quad w'' = -0.88$$

$$L(0) = 4, \quad L(4.4) = 5.76, \quad L(-0.88) = 8.2944$$

El tamaño del paso produce oscilaciones crecientes.

Una tasa demasiado pequeña, en cambio, puede exigir muchas actualizaciones.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Regla de la cadena

Para una neurona con $z = wx + b$, $\hat{y} = \sigma(z)$ y pérdida cuadrática:

$$\frac{\partial L}{\partial w} = \underbrace{(\hat{y} - y)}_{\partial L / \partial \hat{y}} \underbrace{\hat{y}(1 - \hat{y})}_{\partial \hat{y} / \partial z} \underbrace{x}_{\partial z / \partial w}$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

La cadena conecta el efecto de un peso con la pérdida final.

Fuente: capítulo 9, pp. 308–313.

La derivada de la sigmoide

$$\begin{aligned}\sigma(z) &= (1 + e^{-z})^{-1} \\ \sigma'(z) &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right) \\ &= \sigma(z)(1 - \sigma(z))\end{aligned}$$

En $z = 0$, la derivada vale 0.25. En extremos de z , se aproxima a cero.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Gradiente del sesgo

Para $z = \sum_i w_i x_i + b$:

$$\frac{\partial z}{\partial b} = 1$$

$$\frac{\partial L}{\partial b} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

$$b' = b - \eta \frac{\partial L}{\partial b}$$

El sesgo recibe el mismo error local de la neurona sin multiplicarlo por una entrada.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Retropropagación en la red de dos capas

Con sigmoide en ambas capas y $L = \frac{1}{2}(\hat{y} - y)^2$:

$$\delta^{(2)} = (\hat{y} - y)\hat{y}(1 - \hat{y})$$

$$\nabla_{W^{(2)}} L = \delta^{(2)}(a^{(1)})^T, \quad \nabla_{b^{(2)}} L = \delta^{(2)}$$

$$\delta^{(1)} = ((W^{(2)})^T \delta^{(2)}) \odot a^{(1)} \odot (1 - a^{(1)})$$

$$\nabla_{W^{(1)}} L = \delta^{(1)}x^T, \quad \nabla_{b^{(1)}} L = \delta^{(1)}$$

$\odot$ significa multiplicación elemento a elemento. Los gradientes usan los pesos anteriores a la actualización.

Fuente: capítulo 9, pp. 309–313. Formulación matricial con sesgos.

Una actualización de los pesos de salida

Retomamos $\hat{y} = 0.4252$, $y = 1$, $a_1 = 0.3775$, $a_2 = 0.6792$ y $\eta = 0.1$.

$$\delta^{(2)} = (0.4252 - 1)(0.4252)(1 - 0.4252) \approx -0.1405$$

$$\frac{\partial L}{\partial w_1} \approx -0.0530, \quad \frac{\partial L}{\partial w_2} \approx -0.0954$$

$$w'_1 = 1.0053, \quad w'_2 = -0.9905, \quad b' = 0.0141$$

Si actualizamos solo la salida y conservamos la capa oculta para aislar el efecto, la predicción aumenta aproximadamente a 0.431. En entrenamiento completo también actualizamos la capa oculta.

Error local de las neuronas ocultas

Usamos los pesos de salida antes de actualizarlos:

$$\delta_j^{(1)} = w_j^{(2)} \delta^{(2)} a_j^{(1)} (1 - a_j^{(1)})$$

$$\delta_1^{(1)} = (1)(-0.14049)(0.37754)(1 - 0.37754) \approx -0.03302$$

$$\delta_2^{(1)} = (-1)(-0.14049)(0.67918)(1 - 0.67918) \approx 0.03061$$

Los signos distintos reflejan la influencia de cada neurona sobre la salida.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Gradientes de todos los pesos de entrada

Cada error local se multiplica por cada entrada:

$$\begin{aligned}\nabla_{W^{(1)}} L &= \delta^{(1)} x^T \\ &= \begin{bmatrix} -0.03302 \\ 0.03061 \end{bmatrix} \begin{bmatrix} 0.5 & 1 \end{bmatrix} = \begin{bmatrix} -0.01651 & -0.03302 \\ 0.01531 & 0.03061 \end{bmatrix}\end{aligned}$$

La matriz del gradiente tiene exactamente la forma de la matriz de pesos.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Actualización completa de la red

Con $\eta = 0.1$:

$$W_{nuevo}^{(1)} = \begin{bmatrix} 1.00165 & -0.99670 \\ 0.49847 & 0.49694 \end{bmatrix}, \quad b_{nuevo}^{(1)} = \begin{bmatrix} 0.00330 \\ -0.00306 \end{bmatrix}$$

$$W_{nuevo}^{(2)} = \begin{bmatrix} 1.00530 & -0.99046 \end{bmatrix}, \quad b_{nuevo}^{(2)} = 0.01405$$

Todos los gradientes se calculan antes de modificar cualquier parámetro.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Nueva propagación y comprobación de la pérdida

Con todos los parámetros actualizados:

$$\hat{y}_{\text{antes}} = 0.425157, \quad \hat{y}_{\text{después}} = 0.431469$$

$$L_{\text{antes}} = 0.165222, \quad L_{\text{después}} = 0.161614$$

La pérdida de este ejemplo disminuye después del paso.

Esta comprobación local no demuestra mejora en datos nuevos.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Verificación de un gradiente por diferencias finitas

Para un parámetro θ_i y un ε pequeño:

$$g_i^{\text{num}} \approx \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon}$$

Se compara con el gradiente analítico calculado por retropropagación.

Si la discrepancia es grande, revisar signos, dimensiones, transpuestas y evaluación de activaciones.

Esta prueba se usa para comprobar implementaciones pequeñas, no para entrenar redes grandes.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Pseudocódigo del entrenamiento

```
inicializar W1, b1, W2, b2
para cada epoca:
    para cada lote de entrenamiento:
        calcular z1, a1, z2, y_pred
        calcular perdida y gradientes
        W1 = W1 - tasa * grad_W1
        b1 = b1 - tasa * grad_b1
        W2 = W2 - tasa * grad_W2
        b2 = b2 - tasa * grad_b2
    evaluar perdida de validacion
    comprobar criterio de parada
```

Fuente: capítulo 9, pp. 313–314. Adaptación con lotes y validación.

Épocas, lotes y tasa de aprendizaje

- Una época recorre todo el conjunto de entrenamiento.
- Un lote contiene los ejemplos usados para calcular una actualización.
- Descenso por lote completo: todos los ejemplos por actualización.
- Descenso estocástico: un ejemplo por actualización.
- Minilotes: subconjuntos de ejemplos. La tasa de aprendizaje controla el tamaño del ajuste.

Fuente: capítulo 9, pp. 305, 314 y 318. Distinciones complementarias.

Promedio de gradientes en un minilote

Para un lote de B ejemplos:

$$L_B = \frac{1}{B} \sum_{n=1}^B L_n$$

$$\nabla_{\theta} L_B = \frac{1}{B} \sum_{n=1}^B \nabla_{\theta} L_n$$

La actualización usa el promedio. Una época puede contener muchas actualizaciones.

Con 100 ejemplos y lotes de 20, hay cinco actualizaciones por época.

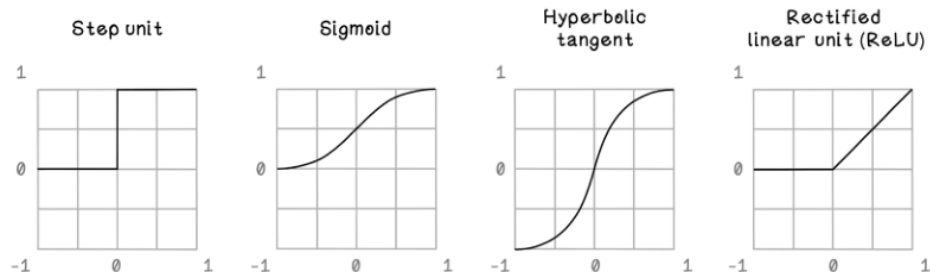
Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Retropropagación y optimizador

- La retropropagación calcula derivadas de la pérdida respecto a parámetros.
- El optimizador utiliza esos gradientes para decidir la actualización.
- El descenso del gradiente es una regla de actualización sencilla.
- El muestreo estocástico introduce ruido y puede modificar la trayectoria. No garantiza alcanzar el mínimo global.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Funciones de activación del capítulo



Escalón, sigmoide, tangente hiperbólica y ReLU.

La gráfica muestra una ventana del dominio. Sigmoide, tanh y ReLU admiten entradas reales fuera de esa ventana.

Fuente: capítulo 9, figura 9.29, p. 315.

Propiedades de las activaciones

Función	Expresión	Propiedad relevante
Escalón	$1[z \geq 0]$	Derivada cero casi en todas partes. Dificulta entrenar por gradiente.
Sigmoide	$1/(1 + e^{-z})$	Salida entre 0 y 1. Se satura en los extremos.
Tanh	$\tanh(z)$	Salida entre -1 y 1 . También puede saturarse.
ReLU	$\max(0, z)$	Salida no negativa. Gradiente 1 para $z > 0$ y 0 para $z < 0$.

Sin activaciones no lineales, una composición de capas afines equivale a una sola transformación afín.

Fuente: capítulo 9, pp. 314–315. Precisiones matemáticas.

Saturación de la sigmoide

z	$\sigma(z)$	$\sigma'(z)$
0	0.50000	0.25000
2	0.88080	0.10499
5	0.99331	0.00665
10	0.99995	0.00005

Una salida extrema puede tener una derivada muy pequeña. Encadenar varias derivadas pequeñas reduce la señal de aprendizaje.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Tangente hiperbólica

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh^2(z)$$

La salida pertenece a $(-1, 1)$ y está centrada en cero.

En $z = 0$, la derivada vale 1. Para magnitudes grandes, se aproxima a cero.

Su conveniencia depende del problema y la configuración.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

ReLU y regiones sin gradiente

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 0 & z < 0 \\ 1 & z > 0. \end{cases}$$

En cero la derivada no existe. Las implementaciones adoptan una convención.

Para entradas negativas, la neurona produce cero. Si permanece allí para todos los ejemplos, puede dejar de recibir actualizaciones útiles.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Gradientes que se desvanecen o crecen

- Al multiplicar muchas derivadas pequeñas, el gradiente puede acercarse a cero.
- La saturación de sigmoide y tanh favorece este problema.
- Gradientes demasiado grandes pueden volver inestable la actualización.
- La activación, la profundidad y la inicialización influyen en la propagación del gradiente.
- Una ReLU puede dejar de aprender si permanece en la región negativa.

Fuente: capítulo 9, pp. 315 y 317. Observación complementaria sobre ReLU.

Decisiones para diseñar una red

Decisión	Relación con la tarea
Entradas	Variables disponibles al predecir
Salidas	Objetivo y tipo de predicción
Capas y neuronas	Capacidad para representar relaciones
Activaciones	No linealidad y comportamiento del gradiente
Inicialización y sesgos	Punto de partida y flexibilidad
Pérdida y tasa	Objetivo y dinámica del entrenamiento

Fuente: capítulo 9, pp. 316–318.

Salida y pérdida según el problema

Problema	Salida habitual	Pérdida habitual
Regresión	Lineal	Error cuadrático
Clasificación binaria	Una sigmoide	Entropía cruzada binaria
Clases excluyentes	Softmax con K salidas	Entropía cruzada multiclase

$$L_{\text{binaria}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

La decisión discreta se obtiene después de la salida diferenciable.

Fuente: capítulo 9, pp. 317–318. Softmax y entropía cruzada como complemento.

Softmax y clasificación de dígitos

Para K clases excluyentes:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad \sum_k p_k = 1$$

Una imagen de 16×16 píxeles tiene 256 entradas al aplanarla.

La salida puede tener 10 neuronas para dígitos de 0 a 9.

La clase elegida corresponde al mayor p_k . Para estabilidad se puede restar $\max_j z_j$ antes de exponenciar.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Sigmoide y entropía cruzada binaria

$$L = -[y \log p + (1 - y) \log(1 - p)], \quad p = \sigma(z)$$

Aplicando la regla de la cadena:

$$\frac{\partial L}{\partial z} = p - y$$

Este gradiente difiere del obtenido con pérdida cuadrática:

$$(p - y)p(1 - p).$$

La función de pérdida determina el gradiente. No se puede cambiar la pérdida sin adaptar la retropropagación.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Inicialización y criterio de parada

- Pesos diferentes rompen la simetría entre neuronas de una misma capa.
- La escala de la inicialización debe ser compatible con la activación.
- Los sesgos también son parámetros entrenables.
- Un límite de épocas acota el entrenamiento.
- La parada temprana conserva los parámetros con mejor desempeño de validación.

Fuente: capítulo 9, pp. 305 y 317–318. Parada temprana como complemento.

El problema de inicializar pesos idénticos

- Neuronas con pesos y sesgos idénticos producen la misma salida.
- Si reciben el mismo gradiente, continúan aprendiendo la misma transformación.
- La inicialización aleatoria rompe esa simetría.
- Inicializar todos los sesgos en cero no impone esa simetría cuando los pesos son diferentes.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Ancho, profundidad y número de parámetros

Para 4-5-1 hay 31 parámetros.

Para 4-5-3-1:

$$\# \text{pesos} = 4(5) + 5(3) + 3(1) = 38$$

$$\# \text{sesgos} = 5 + 3 + 1 = 9$$

$$\# \text{parámetros} = 47$$

Más capacidad puede modelar relaciones adicionales, pero también aumentar costo computacional y riesgo de sobreajuste.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Generalización y evaluación

- Una pérdida baja en entrenamiento puede coexistir con errores en datos nuevos.
- Más capas y más épocas no garantizan mejor generalización.
- Para clasificación, revisar matriz de confusión, precisión y sensibilidad.
- Comparar con un modelo sencillo y mantener la prueba separada del ajuste.
- En el caso de colisiones, un falso negativo merece atención por el objetivo de la tarea.

Fuente: capítulo 9, pp. 303, 307 y 318. Criterios de evaluación complementarios.

Diagnóstico a partir de entrenamiento y validación

Entrenamiento	Validación	Hipótesis a revisar
Pérdida alta	Pérdida alta	Capacidad, preparación u optimización
Pérdida baja	Pérdida alta	Sobreaajuste o cambio de distribución
Pérdida baja	Pérdida baja	Comprobar en prueba independiente

Son indicios para investigar, no diagnósticos automáticos.

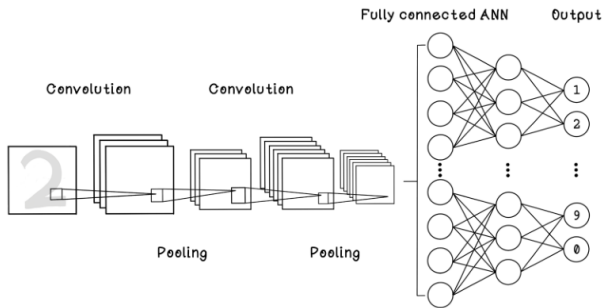
Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Un experimento de comparación

- Comparar una red 4–5–1 con una 4–8–1 usando la misma separación de datos.
- Mantener constantes escalamiento, pérdida y criterio de evaluación.
- Repetir con varias semillas y registrar variabilidad.
- Medir desempeño en validación, tiempo y número de parámetros.
- Seleccionar la configuración antes de evaluar la prueba final.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Redes convolucionales: CNN



La convolución aplica filtros locales con pesos compartidos. El pooling resume regiones y reduce resolución. Aplicaciones del capítulo: clasificación de imágenes y reconocimiento de caracteres.

Fuente: capítulo 9, figura 9.31, p. 319.

Convolución: un filtro local compartido

Un filtro recorre regiones de una imagen y combina sus valores:

$$s_{i,j} = \sum_{u,v} K_{u,v} X_{i+u,j+v} + b$$

El mismo filtro utiliza los mismos pesos en todas las posiciones.

Ejemplo didáctico para un parche $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ y filtro $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$:

$$s = 1(1) + 0(2) + 0(3) - 1(4) = -3.$$

La operación habitual en bibliotecas usa correlación cruzada, sin invertir el filtro.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Pooling y representación de una imagen

Max pooling toma el mayor valor de una región:

$$\begin{bmatrix} 1 & 5 \\ 2 & 4 \end{bmatrix} \mapsto 5$$

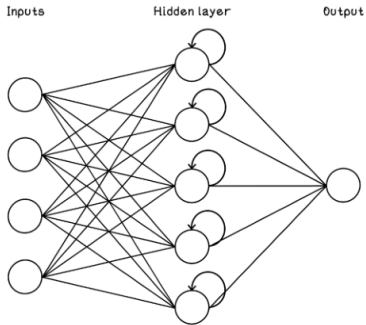
Average pooling produce $(1 + 5 + 2 + 4)/4 = 3$.

El pooling reduce resolución espacial. Las capas posteriores trabajan con mapas de características más compactos.

La convolución aprende filtros. El pooling básico no aprende pesos.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Redes recurrentes: RNN



El estado oculto incorpora información de pasos anteriores:

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b).$$

Los mismos pesos se reutilizan en el tiempo. Procesan secuencias como texto o señales. El entrenamiento propaga gradientes a través de los pasos temporales.

Fuente: capítulo 9, figura 9.32, p. 320.

Estado recurrente en dos pasos

Ejemplo escalar didáctico:

$$h_t = \tanh(0.5x_t + 0.8h_{t-1}), \quad h_0 = 0$$

Para $x_1 = 1$ y $x_2 = 0$:

$$h_1 = \tanh(0.5) \approx 0.4621$$

$$h_2 = \tanh(0.8 \cdot 0.4621) \approx 0.3537$$

La entrada actual vale cero, pero el estado conserva influencia del paso anterior.

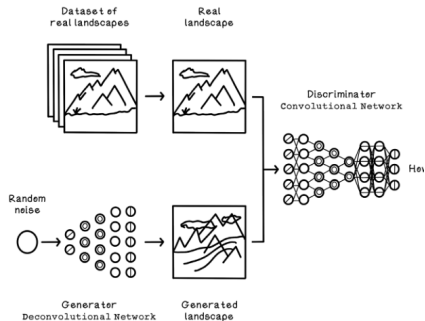
Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Retropropagación a través del tiempo

- La red recurrente se considera desplegada sobre los pasos de la secuencia.
- El gradiente de un peso compartido acumula contribuciones de varios pasos.
- Las derivadas encadenadas pueden desvanecerse o crecer.
- La duración de la secuencia y la dinámica recurrente afectan el entrenamiento.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Redes generativas adversarias: GAN



El generador transforma ruido en ejemplos sintéticos. El discriminador aprende a distinguir ejemplos reales y generados. Ambas redes aprenden mediante objetivos en competencia. El capítulo presenta generación de imágenes y posibles usos engañosos.

Fuente: capítulo 9, figura 9.33, p. 321.

Los objetivos en competencia de una GAN

El discriminador D recibe ejemplos reales o generados por G .

$$\max_D \mathbb{E}_{x \sim p_{\text{datos}}} [\log D(x)] + \mathbb{E}_z [\log(1 - D(G(z)))]$$

Una pérdida frecuente para el generador es:

$$\min_G -\mathbb{E}_z [\log D(G(z))]$$

Se alternan actualizaciones. El discriminador intenta distinguir y el generador intenta producir ejemplos que el discriminador considere reales.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Limitaciones y uso responsable de GAN

- El equilibrio entre generador y discriminador puede ser inestable.
- El generador puede producir poca diversidad aunque algunas muestras parezcan convincentes.
- Los datos sintéticos deben evaluarse por calidad y utilidad para la tarea.
- La generación de contenido plantea preguntas sobre consentimiento, autenticidad y sesgos de los datos.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Arquitectura y estructura de los datos

Arquitectura	Estructura que aprovecha	Ejemplo
Red densa	Vector de características	Clasificar colisiones
CNN	Relaciones locales espaciales	Reconocer un dígito
RNN	Dependencia temporal	Procesar una secuencia
GAN	Generación por competencia	Crear imágenes sintéticas

Estas familias ilustran distintas decisiones de diseño. La evaluación con datos determina si una configuración resulta adecuada.

Fuente: capítulo 9, pp. 319–321.

Ejercicios de la segunda sesión

- 1 Obtener $\delta^{(2)}$ para $p = 0.8$, $y = 1$ y pérdida $\frac{1}{2}(p - y)^2$.
- 2 Actualizar $w = 0.5$ con entrada 0.4 y $\eta = 0.1$.
- 3 Contar los parámetros de 4–8–3–1 con sesgos.
- 4 Explicar qué puede ocurrir si baja la pérdida de entrenamiento y sube la de validación.
- 5 Justificar CNN, RNN o GAN para tres problemas propuestos por el grupo.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Soluciones de la segunda sesión

- ① $\delta^{(2)} = (0.8 - 1)(0.8)(0.2) = -0.032$.
- ② $\partial L / \partial w = (-0.032)(0.4) = -0.0128$, por tanto $w' = 0.50128$.
- ③ Pesos: $32 + 24 + 3 = 59$. Sesgos: $8 + 3 + 1 = 12$. Total: 71.
- ④ Posible sobreajuste. Revisar datos, capacidad y criterio de parada.
- ⑤ Imágenes: CNN. Secuencias: RNN. Generación por competencia: GAN.

Fuente: capítulo 9, Complemento didáctico al capítulo 9.

Cierre: el entrenamiento como problema de optimización

- La pérdida expresa el objetivo que queremos mejorar.
- La retropropagación calcula cómo afectan los parámetros a ese objetivo.
- El optimizador determina los pasos de actualización.
- La validación comprueba si el proceso produce una configuración útil para datos nuevos.

Referencia y alcance

Fuente principal

Capítulo 9, “Artificial neural networks”, del libro guía *Grokking Artificial Intelligence Algorithms*. Archivo proporcionado: `red_neuronal.pdf`, pp. 279–322.

Las figuras incluidas proceden del capítulo, con atribución en sus diapositivas.

Complementos didácticos

Red numérica 2–2–1, XOR, notación matricial con sesgos, distinción entre error y pérdida, separación de validación y salidas habituales por tarea.

Los ejemplos del capítulo tienen propósito pedagógico.